

3.2.2 生成控制

生成控制是调节生成器输出的另一种方法，旨在提高生成文本的多样性和相关性。通过引入特定的控制机制，可以引导生成器产生满足特定条件的文本，如风格、长度或者包含特定信息。生成控制的技术包括：

1. **温度调节**：调整生成过程中的温度参数，以控制文本的创新性和多样性。
2. **最大长度和提前终止**：设定生成文本的最大长度，以及在满足特定条件时提前终止生成，确保输出的紧凑性和相关性。
3. **控制词汇**：通过强制包含或排除特定词汇，引导文本生成满足特定要求。
4. **条件生成**：利用额外的输入信息（如问题类型或领域标签）来引导生成过程，以产生更加相关的回答。

六、RAG 评估

1. 评估方法

评估RAG有效性主要采用两种方法：独立评估和端到端评估。这两种方法从不同角度检验RAG系统的效能，确保其在各个组成部分均表现出色。

1.1 独立评估

独立评估侧重于对RAG系统中检索模块和生成模块的单独评估，以确保每个模块都能有效地完成其任务。

工具推荐：

[scikit-learn](#)

[ranx](#)

[RecTools](#)

1.1.1 检索模块评估

- 检索模块的评估通常借助于度量系统（如搜索引擎、推荐系统或信息检索系统）的效能指标来进行。这些指标衡量了系统在给定查询或任务下的项目排名效能，包括命中率（Hit Rate）、平均排名倒数（MRR）、归一化折损累积增益（NDCG）、精度（Precision）等。

(1) 命中率 (Hit Rate)

命中率是衡量检索系统能否返回相关结果的简单而直接的指标。它通常定义为系统返回的相关文档数量与查询的总次数的比例。命中率高意味着用户更有可能在首次查询时就获得相关结果。

计算方法：命中率 = $\frac{\text{查询检索到至少一个相关文档的次数}}{\text{总查询次数}}$

>>> 举例讲解

假设我们有一个简单的检索系统和以下的查询记录：

- 总共有10次查询。
- 在这10次查询中，有7次查询检索到了至少一个相关的文档。

在这个例子中，命中率可以这样计算：

$$\text{命中率} = \frac{\text{查询检索到至少一个相关文档的次数}}{\text{总查询次数}} = \frac{7}{10} = 0.7$$

这意味着在所有查询中，有70%的查询至少检索到了一个相关文档，显示出检索系统对用户查询有较好的响应能力。

(2) 平均排名倒数 (Mean Reciprocal Rank, MRR)

MRR是一个评估信息检索系统以及问答系统性能的指标，尤其是在只关心最高排名结果的情境下。MRR是所有查询的排名倒数的平均值。

计算方法： $MRR = \frac{1}{N} \sum_{i=1}^N \frac{1}{rank_i}$

其中， N 是查询的总数， $rank_i$ 是第 i 个查询返回的第一个相关文档的排名。

>>> 举例讲解

假设我们有三个查询，它们找到第一个相关文档的排名分别是：

- 查询1：第一个相关文档排在第1位
- 查询2：第一个相关文档排在第3位
- 查询3：第一个相关文档排在第2位

为了计算MRR，我们首先计算每个查询的排名倒数，然后计算这些倒数的平均值：

- 查询1的排名倒数： $1/1 = 1.0$
- 查询2的排名倒数： $1/3 \approx 0.333$
- 查询3的排名倒数： $1/2 = 0.5$

MRR计算公式如下：

$$MRR = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{排名}_i}$$

其中，N是查询数量，排名_i 是第i个查询找到第一个相关文档的排名。

根据上述例子，MRR计算如下：

$$MRR = \frac{1}{3}(1.0 + 0.333 + 0.5) \approx 0.611$$

```
def mean_reciprocal_rank(rankings):  
    """  
    计算平均排名倒数（MRR）。  
  
    参数：  
    - rankings: 一个列表的列表，每个内部列表包含单个查询的相关项的排名，假设排名从1开始。  
  
    返回：  
    - MRR值。  
    """  
    rr_sum = 0.0  
    for rank in rankings:  
        if rank:  
            rr_sum += 1.0 / min(rank)  
    return rr_sum / len(rankings) if rankings else 0  
  
# 测试MRR计算  
rankings = [  
    [1, 2, 3], # 第一个查询，相关项在第一位  
    [3, 4], # 第二个查询，相关项在第三位  
    [2, 3, 4], # 第三个查询，相关项在第二位  
]  
  
mrr = mean_reciprocal_rank(rankings)  
print(f"MRR: {mrr:.3f}")  
  
'''输出值  
MRR: 0.611  
'''
```